

1. Strings

a) Strings basics

```
In [4]: # --- 1. STRINGS ---
text = " Python Programming "
print(text.strip())           # Output: Python Programming
print(text.lower())          # Output: python programming
print(text.upper())          # Output: PYTHON PROGRAMMING
print(text.replace("Python", "Java")) # Output: Java Programming
print(text.strip().split(" ")) # Output: ['Python', 'Programming']
print(len(text))             # Output: 22
print(text.strip().startswith("Py")) # Output: True

print("\n-----\n")

# String Slicing
lang = "Python"
print(lang[0:3])             # Output: Pyt
print(lang[-1])              # Output: n
print(lang[::-1])            # Output: nohtyP
```

```
Python Programming
python programming
PYTHON PROGRAMMING
Java Programming
['Python', 'Programming']
22
True
```

```
-----

Pyt
n
nohtyP
```

b) Mostly used

Whitespace Removal

```
In [27]: dirty_text = " Data Science \n"
print(dirty_text.strip()) # Output: removes both left and right whitespace
```

```
Data Science
```

Case Normalization

```
In [28]: category = " machiNe leaRning "
print(category.strip().lower()) # Output: machine learning
```

```
print(category.strip().upper())    # Output: MACHINE LEARNING
print(category.strip().title())    # Output: Machine Learning
```

```
machine learning
MACHINE LEARNING
Machine Learning
```

Character Replacement & Removal

```
In [30]: price = "$1,250.50"
cleaned_price = price.replace("$", "").replace(",", "")
print(cleaned_price)                # Output: 1250.50
print(float(cleaned_price))         # Output: 1250.5
```

```
1250.50
```

```
1250.5
```

Splitting and Parsing

```
In [33]: email = "user.name@company.com"
print(email.split("@"))             # Output: ['user.name', 'company.com']
print(email.split("@")[1])          # Output: company.com

filename = "dataset_v2.csv"
print(filename.endswith(".csv"))    # Output: True
```

```
['user.name', 'company.com']
```

```
company.com
```

```
True
```

2. Lists

a) Basics

```
In [7]: # --- 2. LIST ---
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")             # Adds to end
print(fruits)                       # Output: ['apple', 'banana', 'cherry', 'orange']

fruits.insert(1, "mango")           # Inserts at index 1
print(fruits)                       # Output: ['apple', 'mango', 'banana', 'cherry', 'orange']

fruits.remove("banana")             # Removes first match
print(fruits)                       # Output: ['apple', 'mango', 'cherry', 'orange']

print("\n-----\n")

fruits.pop()                        # Removes last item
print(fruits)                       # Output: ['apple', 'mango', 'cherry']

print(len(fruits))                  # Output: 3
```

```
fruits.sort()           # Sorts alphabetically in place
print(fruits)           # Output: ['apple', 'cherry', 'mango']
```

```
['apple', 'banana', 'cherry', 'orange']
['apple', 'mango', 'banana', 'cherry', 'orange']
['apple', 'mango', 'cherry', 'orange']
```

```
-----

['apple', 'mango', 'cherry']
3
['apple', 'cherry', 'mango']
```

b) Mostly used - List

Filtering Out Invalid Data with List Comprehensions

```
In [34]: raw_scores = [85, -999, 90, None, 78, -999]
clean_scores = [s for s in raw_scores if s is not None and s != -999]
print(clean_scores)           # Output: [85, 90, 78]

[85, 90, 78]
```

Removing Duplicates

```
In [37]: categories = ["tech", "finance", "tech", "health", "FINANCE"]
unique_cats = list(set(categories))           # set in python makes unique.
print(unique_cats)

['tech', 'FINANCE', 'finance', 'health']
```

Flattening Nested Lists

```
In [38]: nested_data = [[1, 2], [3, 4, 5], [6]]
flat_data = [item for sublist in nested_data for item in sublist]
print(flat_data)           # Output: [1, 2, 3, 4, 5, 6]

[1, 2, 3, 4, 5, 6]
```

Safely Parsing Mixed Types

```
In [39]: mixed_values = ["10.5", "20.1", "missing", "35.0"]
numeric_values = []
for val in mixed_values:
    try:
        numeric_values.append(float(val))
    except ValueError:
        numeric_values.append(0.0) # Impute missing/bad values with 0
print(numeric_values)           # Output: [10.5, 20.1, 0.0, 35.0]

[10.5, 20.1, 0.0, 35.0]
```

3. DICTIONARIES

```
In [9]: # --- 3. DICTIONARIES ---
user = {"name": "Alice", "age": 25, "city": "New York"}

print(user["name"])           # Output: Alice
print(user.get("age"))        # Output: 25
print(user.get("country", "USA")) # Output: USA (default if key missing)

print("\n-----\n")

user["email"] = "alice@mail.com" # Adds new key-value pair
print(list(user.keys()))          # Output: ['name', 'age', 'city', 'email']
print(list(user.values()))        # Output: ['Alice', 25, 'New York', 'alice@mail.com']
print(list(user.items()))         # Output: [('name', 'Alice'), ('age', 25), ('city', 'New York'), ('email', 'alice@mail.com')]

user.pop("city")                # Removes key-value pair
print(user)                     # Output: {'name': 'Alice', 'age': 25, 'email': 'alice@mail.com'}
```

Alice

25

USA

```
['name', 'age', 'city', 'email']
['Alice', 25, 'New York', 'alice@mail.com']
[('name', 'Alice'), ('age', 25), ('city', 'New York'), ('email', 'alice@mail.com')]
{'name': 'Alice', 'age': 25, 'email': 'alice@mail.com'}
```

4. Functions

A basic function with return

```
In [17]: def add(a, b):           # a, b are parameters or arguments
          return a + b           # return at last

print(add(3, 5))                 # Using function
```

8

Function with no return

```
In [18]: def add_and_print(a, b):
          print(a + b)

add_and_print(3, 5)              # Using function
```

8

Function for list

```
In [20]: my_list = [1, 2, 3, 4, 5, 6]

def print_even_numbers(numbers):
    for num in numbers:
        if num % 2 == 0:
            print(num)

print_even_numbers(my_list)
```

2
4
6

Function for Dictionary

```
In [22]: def create_student(name, age, grade="A"):    # Uses default grade ("A") if grade n
        student = {
            "name": name,
            "age": age,
            "grade": grade
        }
        return student

print(create_student("Alice", 20))    # Uses default grade ("A")
print(create_student("Bob", 22, grade="B"))    # Overrides default grade
```

```
{'name': 'Alice', 'age': 20, 'grade': 'A'}
{'name': 'Bob', 'age': 22, 'grade': 'B'}
```

In []: